

SMART MINE

Coal-Mine Compliance & Safety Platform with Blockchain Trust Layer

Final Architecture — Document

1. Project Overview

Smart Mine is a coal-mine compliance and safety management platform. The application UI is a React single-page app (Vite, port 3000) that bundles two frontends: the production Security & Compliance module (src/security) and the older MineGov analytical demo (src/App.tsx with component views and services). Operational data is stored off-chain in Firebase (Firestore is the source of truth, Firebase Auth supplies roles, Firebase Storage holds photos/videos). A hybrid blockchain trust layer (src/blockchain, ethers v6) anchors the SHA-256 hash of every critical record onto Solidity ledgers deployed with Hardhat (chainId 31337), yielding tamper-evident, independently verifiable audit trails without moving interactive data on-chain.

Two optional Python microservices complement the stack: an ML server (ml_server.py, :5001) blending historical CSV datasets with live records for risk/anomaly scoring, and a DB server (db_server.py, :5004) that persists legacy demo state to PostgreSQL 15 (Docker). Every layer degrades gracefully, so the app is always functional: with a running chain, without a chain (simulation mode), and fully offline.

Repository Layout

Path	Responsibility
src/security/	Production compliance & safety module (services, views, safety + blockchain UI pages)
src/blockchain/	Hybrid trust layer — pure-TypeScript services (hash, provider, signer, verifier, per-domain)
contracts/	Ten Solidity 0.8.24 ledger contracts (OpenZeppelin AccessControl + ReentrancyGuard)
scripts/deploy.ts	Compile + deploy all ledgers and write contract addresses to .env and deployments.json
test/blockchain.test.ts	Fifteen Hardhat/Chai contract tests (verify, tamper, duplicate-ID, events, ACL)
supabase/	Legacy schema (tables, RLS policies, triggers, storage buckets) — superseded by Firebase
docs/	ARCHITECTURE.md with file/line flow traces + Mermaid diagrams (.mmd)
public/data/	Thirteen CSV demo datasets driving the ML analytical demo
ml_server.py / db_server.py	Python Flask microservices (ML risk; PostgreSQL persistence)

2. System Architecture

Reference Diagram

REACT FRONTEND — Vite SPA · port 3000

- Security & Compliance app: src/security/securityMain.tsx -> auth gate -> role-based routing
- Legacy MineGov demo: src/App.tsx + src/components/views + src/services

OFF-CHAIN DATA — source of truth

- Firebase: Firestore (records), Auth (roles), Storage (media)
- firebaseDb.ts facade: Firestore + localStorage fallback + 5s timeouts
- IndexedDB multi-tab offline persistence

ML SERVER — Flask :5001

pandas + numpy + scikit-learn (IsolationForest); risk band + anomaly scoring

HYBRID TRUST LAYER — src/blockchain/

- canonicalStringify -> SHA-256 -> bytes32 (deterministic)
- ethers v6: BrowserProvider (MetaMask) + JSON-RPC fallback
- verifier.ts -> VERIFIED / TAMPERED / NOT_FOUND
- Simulation mode when no contract addresses are configured

DB SERVER — Flask :5004

SQLAlchemy + psycopg -> PostgreSQL 15 (docker); 503 degrade -> in-memory

BLOCKCHAIN — Hardhat local node · chainId 31337 · RPC 8545

Ten Solidity ledgers (access-control gated): MineRegistry, RoleRegistry, Attendance, Inspection, Complaint, Incident, Document, Compliance, ApprovalWorkflow, AuditTrail

3. Key Architectural Decisions

1. Hybrid, not full on-chain — Firestore stays the interactive source of truth; only cryptographic hashes go on-chain. The ledger provides integrity, while authorization stays in Firebase rules.
2. Two sub-apps in one repo — the production src/security module and the legacy MineGov demo share the Vite build and Firebase SDK but are fully independent.
3. Simulation mode — with no contract addresses in .env, the blockchain layer deterministically mocks every ledger call, so the entire UI works with zero setup (UI shows a SIMULATED LEDGER badge).
4. Graceful degradation everywhere — Flask servers return 503 instead of crashing; the ML client falls back to live-only estimates; the DB client falls back to in-memory; blockchain calls never throw.
5. Offline-first — IndexedDB multi-tab persistence plus a localStorage collection fallback plus 5-second Firestore timeouts, built for unreliable mine-site connectivity.
6. RBAC everywhere — a role-permission matrix (permissions.ts) with server-side role detection in the auth context, mirrored by firestore.rules and the legacy Supabase RLS policies.
7. Event-sourced audit — complaint_events capture every state transition; audit_logs capture admin actions; ledger

events (RecordRegistered/RecordRevoked) are on-chain.

8. Deterministic hashing — canonicalStringify sorts keys and excludes volatile/derived fields so any client recomputes an identical hash for verification.
9. Domain-driven ledgers — one Solidity contract plus one TypeScript service per domain (attendance, inspection, complaint, incident, document, compliance, approval, audit, mine, role).
10. Blended ML — predictions always combine historical datasets (CSV) with live runtime records in an explicit 0.5/0.5 blend returned by the API for transparency.

4. Workflows

4.1 Runtime Workflow — Complaints (state machine, full chain of custody)

Report Issue -> Submitted -> auto-assigned with SLA due date -> Assigned -> linked Inspection -> CAPA created -> Escalated when overdue/critical (48h SLA) -> Verified -> Resolved -> Archived. Every transition appends to complaint_events; every admin action to audit_logs; critical/immediate-danger reports are flagged is_critical and auto-escalated. Roles gate each step: worker -> mining_mate -> overman -> safety_officer -> mine_manager -> super_admin, each seeing only their scoped views, with realtime notifications via Firestore onSnapshot.

4.2 Blockchain Anchoring Flow

Create record in Firestore -> per-domain service (e.g. complaintBlockchain.ts) -> canonicalStringify -> SHA-256 -> bytes32. If contract addresses are configured, the local Hardhat ledger records the hash via ethers (MetaMask signing / JSON-RPC); otherwise simulation mode returns a deterministic mock. Blockchain metadata (hash, tx, block, status) is written back to Firestore. The Verify Records page recomputes the hash and compares it with the ledger (VERIFIED / TAMPERED); the QR page encodes a verification payload for field verification on any device. Failures are never thrown — they render as FAILED / PENDING badges.

4.3 ML Risk Prediction

Historical CSV datasets (13 files: mines, compliances, inspections, violations, corrective actions, incidents, environment, sensors, equipment, contractors, workers, production, documents) are blended with live runtime records at 0.5/0.5, producing a predicted risk score and band (Low / Moderate / High / Critical), sorted per mine with explicit contributors; the client falls back to live-only estimates if the server is down.

4.4 Development Workflow

Step	Command
Install dependencies	npm install
Frontend dev server (port 3000)	npm run dev
Start local blockchain node	npm run blockchain:start
Deploy 10 contracts + write addresses	npm run blockchain:deploy
Run 15 contract tests	npm run blockchain:test
Start ML server (:5001)	python ml_server.py
Start DB server (:5004)	python db_server.py
Start PostgreSQL 15	docker compose up -d
Run both ML + Vite together	npm start
Production build	npm run build
Deploy to Firebase Hosting	firebase deploy

5. Technology Stack

5.1 Frontend

Layer	Technology
UI framework	React 18 + TypeScript 5.3 (ESM project)
Build tooling	Vite 5 + @vitejs/plugin-react; multi-page (index / security / legacy .html)
Styling	Tailwind CSS 3.4 + PostCSS / Autoprefixer (dark copper theme)

Charts	Recharts
Maps / GIS	Leaflet + react-leaflet
3D visualization	three + @react-three/fiber + @react-three/drei
Utilities	framer-motion, lucide-react, papaparse (CSV), jsPDF (PDF), pdfjs-dist, qrcode.react, tesseract.js (OCR)
Backend SDKs	firebase 10 (Auth / Firestore / Storage)
Blockchain SDK	ethers v6

5.2 Backend & Services

Piece	Technology
Off-chain data	Firebase Firestore (+ rules) and Firebase Storage (+ rules)
Legacy relational	Supabase schema + RLS (supabase/schema.sql) — replaced by Firebase
ML microservice	Python Flask + pandas + numpy + scikit-learn (IsolationForest) — ml_server.py :5001
Persistence microservice	Python Flask + SQLAlchemy 2 + psycopg — db_server.py :5004
Relational database	PostgreSQL 15 (docker-compose)
Smart contracts	Solidity 0.8.24 + OpenZeppelin (AccessControl, ReentrancyGuard) — 10 ledgers
Blockchain tooling	Hardhat 2.29 (local node, chainId 31337), ethers v6, hardhat.config.cjs (CJS for ESM project)

5.3 Testing & Tooling

Area	Stack
Contract tests	Hardhat + Chai — 15 tests (verify / tamper / duplicate-ID / events / ACL)
Type checking	tsc (npm run lint)
E2E / browser automation	Playwright 1.63 (devDependency)
Process management	concurrently (npm start)
Python dependencies	flask, pandas, numpy, scikit-learn, sqlalchemy, psycopg

6. Engineering Methodology

Smart Mine follows ten engineering principles, consistently visible across the codebase:

1. Hybrid trust / hash-anchoring — integrity on-chain, performance off-chain. The blockchain layer provides integrity, never authorization.
2. Simulation-first development — every blockchain feature works with zero setup so the UI can be demoed with no chain and no keys.
3. Graceful degradation — any single dependency (chain, ML server, DB server) can be down without breaking the app.
4. Offline-first design — IndexedDB multi-tab persistence, localStorage fallback and 5s operation timeouts for flaky mine-site networks.
5. RBAC everywhere — role-permission matrix, server-side role detection and mirrored rules in firestore.rules and legacy RLS policies.
6. Event-sourced auditing — complaint_events and audit_logs keep an immutable chain of custody; deletions are audited first.
7. Deterministic verification — canonicalStringify yields byte-identical hashes across any client for trustworthy verification.
8. Domain-driven ledgers — one contract plus one service per domain keeps the trust layer symmetric and testable.
9. Deterministic contract testing — one test pattern per concern across all ten ledgers (register / verify / tamper / duplicate / events / ACL).
10. Data-driven blended ML — predictions always combine historical datasets with live records and expose the blend ratios in the API.